

The architectural shift

Onion · MediatR · pipelines · versioning · `IExceptionHandler`

TMS

```
await mediator.Send(cmd, ct);
```

```
[ApiVersion("1.0")]
```

```
[Route("api/v{version:apiVersion}/courses" )]
```

The messenger and the chef

Session goals

By the end of this session, you will:

- Implement Onion Architecture business rules isolated from infrastructure
- Master MediatR dispatching controllers stay messengers, handlers stay chefs
- Apply pipeline behaviors for validation and logging without duplication
- Separate Commands (writes) from Queries (reads) in the CQRS sense
- Configure API versioning so V1 tablets and V2 web coexist
- Standardize failures with `ExceptionHandler` and `ProblemDetails`

The TMS scaling crisis

- M6 API works, passes tests, ships then 200 students hit it the same morning
- Database CPU hits 100%; responses climb to seconds; admin dashboard times out manager calls
- Caching prevents database meltdowns
- Versioning prevents breaking deployed clients
- Resilience prevents cascading outages
- Architecture prevents unmaintainable spaghetti code

Why Module 7 exists

Every topic this week answers a failure mode real production APIs hit urgency before patterns.

Onion Architecture the fortress

- Domain pure C#, zero NuGet; entities, value objects, rules
- Application use cases (MediatR handlers), DTOs, interfaces
- Infrastructure EF Core, Redis, external APIs, files
- Presentation controllers, middleware

Golden rule

Dependencies flow inward Domain never references the database; Application never references Infrastructure. Red flag: Domain references `Microsoft.EntityFrameworkCore`.

MediatR messenger vs chef

Controller messenger

```
[HttpPost]

public async Task<IActionResult> Enroll(
    EnrollStudentCommand cmd, CancellationToken ct)
{
    var result = await mediator.Send(cmd, ct);

    return result.IsSuccess
        ? CreatedAtAction(nameof(GetSchedule),
            new { id = result.EnrollmentId }, result)
        : BadRequest(result.ErrorMessage);
}
```

Handler chef

```
public class EnrollStudentHandler(
    IEnrollmentRepository repo,
    ICourseRepository courses)
    : IRequestHandler<
        EnrollStudentCommand, EnrollmentResult>
{
    public async Task<EnrollmentResult> Handle(
        EnrollStudentCommand command, CancellationToken ct)
    {
        // Business logic isolated, testable, reusable
    }
}
```

Skinny controller rule

The HTTP branch is thin enrollment can be 100 lines in Handle; business if checks do not live in the controller.

Pipeline behaviors quality at scale

ValidationBehavior

```
public class ValidationBehavior<TReq, TRes>(
    IEnumerable<IValidator<TReq>> validators)

    : IPipelineBehavior<TReq, TRes> where TReq : notnull
{
    public async Task<TRes> Handle(
        TReq request, RequestHandlerDelegate<TRes> next,
        CancellationToken ct)
    {
        var failures = validators
            .Select(v => v.Validate(request))
            .SelectMany(r => r.Errors)
            .Where(f => f is not null)
            .ToList();

        if (failures.Count > 0) throw new
            ValidationException(failures);

        return await next();
    }
}
```

- Register once validation (and logging behaviors) run before every handler
- Compare to try/catch + validate copied into fifty controller actions

API versioning safe contract evolution

- Problem: 500 centre tablets on V1; web needs V2 you cannot silently rename V1
- Solution: V1 and V2 routes coexist tablets stay on /api/v1/..., dashboard on /api/v2/...

```
[ApiVersion("1.0")]  
  
[Route("api/v{version:apiVersion}/courses")]  
public class CoursesV1Controller : ControllerBase { }  
  
[ApiVersion("2.0")]  
  
[Route("api/v{version:apiVersion}/courses")]  
public class CoursesV2Controller : ControllerBase { }
```

What happens to tablets in the field when you rename a JSON field on the only URL they know?

IExceptionHandler centralized stability

GlobalExceptionHandler

```
public class GlobalExceptionHandler(
    ILogger<GlobalExceptionHandler> logger)
    : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(
        HttpContext ctx, Exception ex, CancellationToken ct)
    {
        logger.LogError(ex, "Unhandled: {Message}", ex.Message);

        var problemDetails = new ProblemDetails {

            Status = StatusCodes.Status500InternalServerError,

            Title = "Server Error",

            Detail = "An unexpected error occurred. ID: "

                + ctx.TraceIdentifier };

        ctx.Response.StatusCode = problemDetails.Status.Value;

        await ctx.Response.WriteAsJsonAsync(problemDetails, ct);

        return true;
    }
}
```

- Uniform ProblemDetails + TraceIdentifier professionalism under failure
- IExceptionHandler + ExceptionHandlerMiddleware DI-friendly in .NET 10 vs ad-hoc controller try/catch

Lab brief Session 1

- M7-Lab-Workbook Exercises 1 and 2 (full steps there)
- Exercise 1: API versioning V1 and V2 both respond
- Exercise 2: CQRS with MediatR commands, queries, validation pipeline
- Checkpoints: both versions live; controller has zero business rules; pipeline rejects bad commands

Session 1 summary

- Onion protect domain from infrastructure churn
- MediatR decouple HTTP from use-case code
- Pipeline behaviors validation/logging without duplication
- API versioning old and new clients safely coexist
- IExceptionHandler one professional error envelope

Interview (spine wording)

Interviewer: “How do you validate across 50 endpoints?” Answer: “MediatR pipeline behavior + FluentValidation every command validated before the handler runs.”

Next: Session 2 performance & resilience

HybridCache stampede protection, token-bucket fairness, BackgroundService + Channel<T> and 202 Accepted for long TMS jobs.